

Product Requirements Document (PRD): Agentic MLOps Orchestrator v1.0

1. Product Vision & Problem Statement

The goal is to bridge the **"Configuration Chasm"** that prevents domain experts (pricing analysts, marketing data scientists) from scaling Machine Learning. The core barrier is not the math, but the **orchestration of disparate cloud services**. Current MLOps requires deep knowledge of **Kubeflow Pipelines (KFP)**, **Docker**, and complex **YAML**. This product introduces an **Agentic Infrastructure Bridge** to automate these technical hurdles through natural language.

2. Implementation Model: Agent-as-Orchestrator

The system will prioritize **Interactivity** over static reproducibility.

- **Logic Root:** Managed by the **LLM State Machine** rather than a static KFP DAG.
- **Human-in-the-Loop:** The agent must pause for feedback at every lifecycle step (e.g., confirming column names or infrastructure choices).
- **State Dependency:** The solution relies on the **ADK agent's Session State** to pass credentials and IDs between modular tools.

3. Functional Requirements & Tool Specifications

3.1. MCP Server Interface (server.py)

The MCP Server acts as the execution layer, hosting a suite of **hardened tools** pre-configured for Google Cloud.

Tool Name	Input Parameters	Logic / GCP SDK Call	Output / Agent Action
create_dataset	bq_dataset, bq_table,	aiplatform.TabularDat	Returns ds.resource_name for

	name	dataset.create with bq_source	agent session storage.
train_model	model_name, dataset_id (opt), gcs_uri (opt)	aiplatform.CustomTrainingJob using train.py and TRAIN_IMAGE.	Returns job.resource_name ; supports async monitoring.
deploy_to_vertex	model_id, endpoint_name	aiplatform.Endpoint.create; model.deploy with n1-standard-4.	Returns endpoint.resource_name.
deploy_to_gke	model_id, cluster_name	client.AppsV1Api() to create a V1Deployment with 2 replicas.	Returns GKE Deployment Status.

3.2. Session & State Management

- **Technical ID Storage:** The agent must store **Dataset IDs**, **Model URIs**, and **Job Handles** in its local session.
- **Input/Output Stitching:** The agent must pass outputs from one GCP component as inputs to the next to shield the user from "technical plumbing".
- **Polling:** The agent acts as a persistent state machine to monitor asynchronous cloud-native jobs across days or weeks.

3.3. Infrastructure Flexibility

- **Data Ingestion:** Support both **Vertex Managed Datasets** (for lineage) and **GCS Shards** (for raw data drops).

- **Serving:** Support **Vertex Endpoints** (auto-scaling) or **GKE** (custom runtime flexibility/cross-cluster parity).

4. Enterprise Readiness & Security

4.1. Governance & Monitoring

- **Automated Setup:** The agent must proactively configure **Vertex Model Monitoring** during deployment.
- **Drift Detection:** Automated tracking of data drift and prediction skew.
- **Auditability:** Every step—from ingestion to deployment—must be documented in **Cloud Logging** via the MCP service account.

4.2. Security Architecture

- **IAM Compliance:** The MCP server must respect roles such as `roles/aiplatform.user` and `roles/container.developer`.
- **Network Security:** Strict adherence to **VPC-SC boundaries**.
- **Data Sovereignty:** Training/prediction data is **never pulled** to the agent's local environment; it remains in the GCP execution layer.

5. Detailed User Flow (The "Handshake")

1. **Intent:** User requests a sales forecasting model via NLP.
 2. **Phase 1 (Data):** Agent asks for BigQuery details and confirms ingestion type (Managed Dataset vs GCS).
 3. **Phase 2 (Training):** Agent triggers `train_model` using the ID from Phase 1 and stores the resulting `JOB_ID`.
 4. **Phase 3 (Deployment):** Agent offers GKE vs Vertex; upon user choice, it executes the relevant tool and confirms the live URL.
-

Would you like me to generate the detailed V1Deployment YAML structure and the train.py boilerplate required for the CustomTrainingJob tool?